Setup and Use a Firewall

Windows Defender Firewall & UFW (Linux) — Configuration, Testing, and Validation

PREPARED BY: VIMAL RAJ R

APRIL 2026

Executive Summary

This report presents the configuration and testing of firewall rules on both **Windows Defender Firewall** and **UFW (Uncomplicated Firewall)** on Kali Linux. The objective was to control inbound network traffic by allowing secure services and blocking insecure ones.

Multiple ports were configured to demonstrate real-world firewall behavior — blocking vulnerable services such as Telnet and FTP while permitting secure protocols like SSH and HTTPS.

Goal

Control and filter inbound network traffic using firewall rules

Outcome

Attack surface reduced; unauthorized access effectively mitigated

Objectives & Tools

Objectives

- Configure firewall rules to allow or block specific traffic
- Understand and apply inbound traffic filtering
- Test and validate firewall rules using scanning tools
- Reduce attack surface by enforcing least privilege access

Tools Used

Windows Defender Firewall

Advanced Security GUI (wf.msc) for rule management

UFW

Uncomplicated Firewall on Kali Linux via CLI

Nmap

Network scanning tool used for rule validation

System Environment & Network Setup

Test Systems

System	Role
Windows Machine	Firewall config via GUI (wf.msc)
Kali Linux VM	Firewall config via UFW CLI
Network Type	Local / Internal Network

Network Configuration

Local Testing Environment

All rules were applied and tested within an isolated local network — no external exposure.

Cross-Platform Communication

Windows and Kali Linux VMs communicated internally, allowing real-world firewall behavior to be observed and verified.

Port Configuration Overview

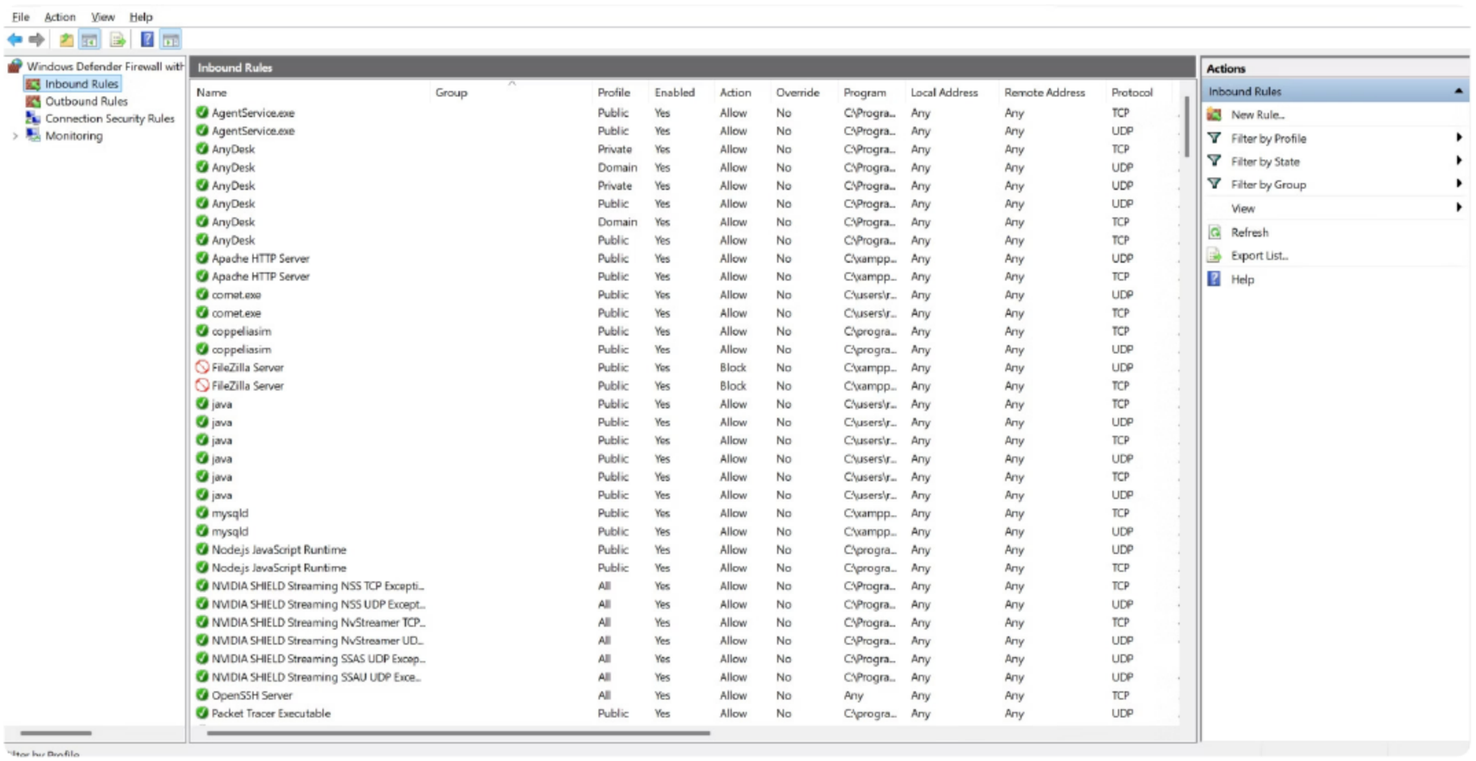
Port	Service	Action	Purpose
21	FTP	Blocked ❌	Prevent insecure file transfer
22	SSH	Allowed ✅	Enable secure remote access
23	Telnet	Blocked ❌	Prevent plaintext login
80	HTTP	Tested ⚠️	Control and monitor web traffic
443	HTTPS	Allowed ✅	Enable secure encrypted communication
3389	RDP	Controlled ⚠️	Remote desktop access (restricted)

ⓘ Security approach: Block all insecure services by default. Allow only explicitly required ports. Apply the **Principle of Least Privilege** throughout.

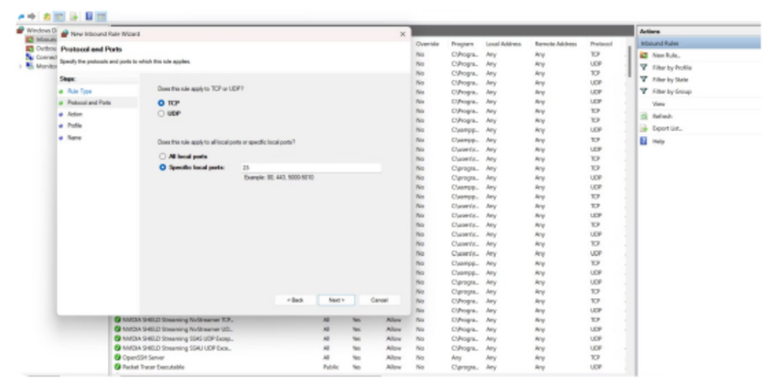
Windows Firewall Configuration

Firewall rules were created using **Windows Defender Firewall with Advanced Security** (wf.msc). Inbound rules were configured to block insecure ports (21, 23) and allow secure services (22, 443, 3389).

Firewall Dashboard — Inbound Rules



New Inbound Rule — Protocol & Port Selection



Applied Inbound Rules Summary

Inbound Rules

Name	Group	Profile	Enabled	Action	Override	Program	Local Address	Remote Address	Protocol
Allow RCP Connections		All	Yes	Allow	No	Any	Any	Any	TCP
Allow HTTPS Connection		All	Yes	Allow	No	Any	Any	Any	TCP
Local HTTP Connection		All	Yes	Block	No	Any	Any	Any	TCP
Allow SSH Connection		All	Yes	Allow	No	Any	Any	Any	TCP
Block FTP Connection		All	Yes	Block	No	Any	Any		
Block Telnet Connection		All	Yes	Block	No	Any	Any		

Linux Firewall Configuration (UFW)

UFW Commands Applied

```
sudo ufw enable
sudo ufw deny 21
sudo ufw deny 23
sudo ufw allow 22
sudo ufw allow 443
sudo ufw deny 80
sudo ufw status numbered
```

UFW was successfully enabled and set to start on system boot. All rules applied for both IPv4 and IPv6.

UFW Enabled & Rules Applied

```
┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw enable
Firewall is active and enabled on system startup

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw status
Status: active

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw status numbered
Status: active

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw deny 23
Rule added
Rule added (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw allow 22
Rule added
Rule added (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw deny 80
Rule added
Rule added (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw allow 443
Rule added
Rule added (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─#
```

UFW Status — Numbered Rule List

```
┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw deny 21
Rule added
Rule added (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─# ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 23                         DENY IN     Anywhere
[ 2] 22                         ALLOW IN    Anywhere
[ 3] 80                         DENY IN     Anywhere
[ 4] 443                        ALLOW IN    Anywhere
[ 5] 21                         DENY IN     Anywhere
[ 6] 23 (v6)                    DENY IN     Anywhere (v6)
[ 7] 22 (v6)                    ALLOW IN    Anywhere (v6)
[ 8] 80 (v6)                    DENY IN     Anywhere (v6)
[ 9] 443 (v6)                   ALLOW IN    Anywhere (v6)
[10] 21 (v6)                    DENY IN     Anywhere (v6)

┌──(root㉿kali)-[/home/vimal/Desktop]
└─#
```

Testing & Validation with Nmap

Nmap Scan Command

```
nmap 127.0.0.1 -p 21,22,23,80,443 -A
```

Scan Results

Port	Status	Result
21 (FTP)	Filtered / Closed	❌ Blocked
22 (SSH)	Open	✅ Allowed
23 (Telnet)	Filtered / Closed	❌ Blocked
80 (HTTP)	Closed	⚠️ Tested
443 (HTTPS)	Open	✅ Allowed

Nmap Scan Output

```
┌──(root㉿kali)-[/home/vimal/Desktop]
└─# nmap 127.0.0.1 -p 21,22,23,80,443 -A
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 09:06 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000040s latency).

PORT    STATE  SERVICE VERSION
21/tcp  closed ftp
22/tcp  open   ssh     OpenSSH 10.2p1 Debian 5 (protocol 2.0)
23/tcp  closed telnet
80/tcp  open   http    Apache httpd 2.4.66 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.66 (Debian)
443/tcp closed https
Device type: general purpose
Running: Linux 5.X|6.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 5.0 - 6.2
Network Distance: 0 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/
 .
Nmap done: 1 IP address (1 host up) scanned in 7.79 seconds

┌──(root㉿kali)-[/home/vimal/Desktop]
└─#
```

Analysis & Key Findings

Before: Risks of Open Ports

🔴 FTP & Telnet Exposed

Plaintext credentials transmitted over the network — easily intercepted

🔴 Unrestricted RDP

Open RDP port is a common target for brute-force and ransomware attacks

🔴 Broad Attack Surface

Every open port is a potential entry point for unauthorized access

After: Security Improvements

✅ Insecure Ports Blocked

FTP (21) and Telnet (23) denied on both Windows and Linux

✅ Least Privilege Applied

Only SSH and HTTPS explicitly permitted; all else restricted

✅ Rules Validated

Nmap confirmed firewall behavior matched configured policy exactly

Conclusion & Recommendations

This exercise demonstrated that properly configured firewall rules — on both Windows Defender Firewall and UFW — are highly effective at controlling network traffic and reducing exposure to cyber threats.

01

Block Insecure Protocols by Default

Deny FTP (21) and Telnet (23) on all systems. These services transmit data in plaintext and should be replaced with SFTP and SSH respectively.

02

Enforce Least Privilege on All Ports

Only allow services that are actively required. Restrict RDP (3389) to specific trusted IPs to minimize brute-force risk.

03

Validate Rules Regularly with Nmap

Periodic scanning confirms that rules behave as intended and no unintended ports have been opened after system changes.

04

Apply Consistent Policy Across Platforms

Maintain equivalent rule sets on both Windows and Linux environments to ensure uniform security posture across the infrastructure.

Both Windows Defender Firewall and UFW proved reliable and effective for enforcing port-based security policies in a local lab environment.